RTL-CLAW

An AI-Agent-Driven Framework for Automated IC Design Flow via OpenClaw and Open-Source EDA Tools

Haotian Yu[1,2], Yuchen Liu[1], Yifan Wang[1], Qibo Xue[3,4], Yuan Pu[3,4], Shuo Yin[3] Yuntao Lu[3], Xufeng Yao[3], Zhuolun He[3,4], Yuyang Ye[1,3,*], Lei Qiu[1], Qing He[1,2], Bei Yu[3]

[1] Tongji University, Shanghai, China

[2] Phlexing Technology Co., Ltd., Hangzhou, China

[3] The Chinese University of Hong Kong, Hong Kong, China

[4] Chateda

Project Leader: ***Yuyang Ye***

eda_tongji@126.com

March 27, 2026

Abstract

RTL-CLAW is an open-source, AI-agent-driven framework for automated IC design flow. Built on OpenClaw and open-source EDA tools, it integrates RTL analysis, testbench generation, RTL optimization, and logic synthesis into a unified workflow. Its core Partition-Opt-Merge strategy decomposes complex RTL into smaller blocks for targeted optimization and then reconstructs optimized designs while preserving functional correctness. RTL-CLAW aims to improve automation, scalability, and usability in open-source RTL design flows.

Contents

Preface ii

1 Introduction 1
- 1.1 Background and Challenges . . . 1
- 1.2 Limitations of Existing Work . . . 1
- 1.3 Solutions of Our Work . . . 1

2 Background 2
- 2.1 Current State of Open-Source EDA Toolchains . . . 2
- 2.2 Large Language Models and AI Agents in EDA . . . 2
- 2.3 Problem Definition . . . 3

3 Methodology 3
- 3.1 System Architecture (RTL-CLAW Framework) . . . 3
- 3.2 RTL Optimization Algorithm Based on Divide-and-Conquer (Partition-Opt-Merge) . . . 3
  - 3.2.1 Verilog-Partition Module . . . 3
  - 3.2.2 Verilog Optimization Module . . . 4
  - 3.2.3 Verilog Merge Module . . . 4
- 3.3 Automated Verification Environment Generation (RTL Testbench Generation) . . . 4
  - 3.3.1 RTL-Spec-Analyzer . . . 4
  - 3.3.2 Testbench-Generator . . . 4
- 3.4 Agent-Based Automated Flow Integration (Agentic Workflow) . . . 5

4 Results 5
- 4.1 Experimental Evaluation . . . 5

5 Conclusion 7
- 5.1 Summary . . . 7
- 5.2 Future Work . . . 7

A Appendix Example 7

Preface

RTL-CLAW is a collaborative open-source project jointly carried out by Tongji University and The Chinese University of Hong Kong (CUHK). The project is led by Yuyang Ye and focuses on building an AI-agent-driven framework for automated IC design flow based on open-source EDA tools.

We would like to express our sincere appreciation to all contributors for their valuable efforts in project planning, system development, experimentation, and documentation. The progress of RTL-CLAW would not have been possible without the collective dedication and collaboration of the team.

This report provides an overview of the project motivation, technical framework, and current implementation status, and is intended to serve as supporting documentation for future development and collaboration.

1 Introduction

This is the main body area of the report. The section files are split into separate `.tex` documents so that you can manage longer projects more comfortably in Overleaf.

A typical introduction usually explains the problem background, the motivation for the report, the scope of the work, and the main contributions or objectives.

1.1 Background and Challenges

With the evolution of Moore's Law, the complexity of integrated circuit (IC) design has grown exponentially. Modern system-on-chips (SoCs) often contain billions of transistors, and their design flow encompasses multiple stages, including register-transfer level (RTL) design, functional verification, logic synthesis, physical design, and sign-off. However, current mainstream design flows suffer from a critical issue known as **toolchain fragmentation**. Although commercial electronic design automation (EDA) tools are powerful, data exchange and process integration between tools from different vendors—or even between different stages within the same vendor's toolchain—often rely heavily on manually written scripts and intermediate files. In the lengthy flow from RTL to GDSII, even minor adjustments to design specifications can lead to re-establishment of the verification environment, modifications to synthesis scripts, and iterations in physical implementation. The entire cycle is lengthy, inefficient, and highly prone to human error. This fragmented flow, which heavily depends on manual intervention, has become a key bottleneck constraining chip design efficiency and time-to-market.

1.2 Limitations of Existing Work

To mitigate the above issues, both industry and academia have developed various scripting languages (e.g., Tcl, Python) and flow management tools (e.g., Makefile) aimed at chaining toolchains together. However, these traditional automation solutions are essentially the execution of "static scripts." They lack the capability for intelligent decision-making: they cannot dynamically adjust optimization strategies based on intermediate results, cannot autonomously respond to unexpected design specification changes, and cannot automatically analyze the causes of simulation errors or attempt to fix them. For instance, a fixed synthesis script cannot determine whether the current design is better suited for area optimization or speed optimization, nor can it automatically attempt an alternative synthesis strategy after an optimization failure. This lack of adaptability and intelligence renders such automation insufficient for meeting the ever-increasing demands of complex chip design.

1.3 Solutions of Our Work

To address the aforementioned challenges, this paper proposes RTL-CLAW, a novel EDA framework based on the OpenClaw AI agent. The main contributions of this paper are as follows:

1. A Fine-Grained RTL Optimization Method. We design and implement a "divide-and-conquer" three-stage optimization strategy, consisting of Partition, Optimization, and Merge. This approach decomposes complex Always blocks (containing mixed logic) that are difficult for traditional synthesis tools to handle into minimal basic blocks. After aggressive optimization,

these blocks are reassembled, significantly improving power, performance, and area (PPA) metrics while maintaining logical equivalence.

2. Automated Verification Environment Setup. Through two core modules—the RTL-Spec-Analyzer and the Testbench-Generator—we achieve automatic generation of a foundational verification platform from design specifications. This drastically reduces the time required for verification environment setup and provides automated support for logic equivalence checking before and after optimization.

3. Full-Flow Agent Orchestration Based on OpenClaw.Utilizing OpenClaw as the core control layer of the AI agent, the entire IC design flow (RTL input, verification, optimization, synthesis) is treated as a series of dynamically schedulable tasks. The agent can autonomously plan the execution sequence, invoke encapsulated tool APIs, and make subsequent decisions based on execution outcomes (e.g., simulation success, synthesis reports), achieving unattended automation from RTL to the front-end of GDSII.

2 Background

2.1 Current State of Open-Source EDA Toolchains

In recent years, the open-source EDA tool ecosystem has flourished, providing robust support for academic research and low-cost design. Yosys is currently the most popular open-source logic synthesis framework, supporting the synthesis of Verilog code into netlists for various target technologies. ABC is a powerful logic optimization and technology mapping tool embedded within Yosys, offering a rich set of Boolean function optimization algorithms. Iverilog (Icarus Verilog) is an efficient Verilog simulator commonly used for functional verification. GTKWave is used for viewing waveform files generated from simulations. Additionally, the OpenROAD project aims to build a complete, open-source, automated chip design platform from RTL to GDSII. However, seamlessly integrating these tools into an automated flow remains challenging. The primary pain point lies in the heterogeneity of data formats: the netlist format output by Yosys, the LEF/DEF physical library formats required by OpenROAD, and constraint files (e.g., SDC) lack standardized automatic conversion and consistency checking mechanisms, often necessitating manual intervention.

2.2 Large Language Models and AI Agents in EDA

The rise of large language models (LLMs) has opened new possibilities for code generation and tool utilization. Existing research has explored using LLMs to assist in generating simple RTL code or test cases. However, upgrading LLMs from “code completion tools” to “agents capable of autonomously performing tasks” represents a more promising direction. OpenClaw is such a framework, endowing AI agents with core control capabilities: Planning ability allows the decomposition of high-level design goals (e.g., “optimize the AES module and complete synthesis”) into a sequence of sub-tasks; Tool Use ability enables the unified invocation of external tools such as Yosys and Iverilog through a standardized interface; Memory ability allows the agent to record key information during execution (e.g., simulation logs, area reports), providing a basis for subsequent decisions. This distinguishes it from static scripts that execute

fixed instructions, granting it the potential to handle complex, dynamic design flows.

2.3 Problem Definition

The core technical problem addressed in this paper can be formally defined as the Complex RTL Optimization Problem. In RTL design, a single ‘always‘ block may contain both sequential logic (e.g., state register updates) and combinational logic (e.g., complex conditional judgments and arithmetic operations). Traditional synthesis tools (such as Yosys’s default optimization flow) often treat such mixed-logic blocks as an indivisible whole during optimization, to conservatively ensure equivalence. This “black-box” approach restricts the granularity of optimization, preventing the tool from performing aggressive constant propagation, resource sharing, or Boolean simplification on the internal logic within the block, thus making it difficult to achieve optimal PPA trade-offs. Safely decoupling, optimizing, and reassembling these complex logic blocks without breaking functional equivalence is key to enhancing design quality.

3 Methodology

3.1 System Architecture (RTL-CLAW Framework)

The architecture of RTL-CLAW is structured into three layers, as illustrated in the conceptual diagram below:

- Interaction Layer (OpenClaw Interface): Provides users with a natural language or command-line interface. Users can input high-level instructions here, such as design specifications, paths to RTL files, and performance constraints.
- Agent Core Layer: This serves as the “brain” of the system.
- Planner: Receives user instructions and decomposes them into an executable chain of tasks (e.g., ‘[“Parse RTL”, “Generate Testbench”, “Simulate and Verify”, “Execute Partition-Opt-Merge Optimization”, “Simulate and Verify Again”, “Execute Logic Synthesis”]‘).
- Tool Executor: Encapsulates the invocation details of all underlying tools (Yosys, ABC, Iverilog, and our custom modules), providing a unified function interface to the Planner and capturing the output logs from the tools.
- Data Flow: The user-input RTL code first flows into the Agent layer. According to the plan, the Agent sequentially passes the data to modules such as ‘testbench-generator‘ and ‘verilog-partition‘, invoking simulation or synthesis tools after each step. Finally, the optimized and synthesized netlist file is output.

3.2 RTL Optimization Algorithm Based on Divide-and-Conquer (Partition-Opt-Merge)

3.2.1 Verilog-Partition Module

This module parses the abstract syntax tree (AST) of Verilog to identify all ‘always‘ blocks. For each complex ‘always‘ block (e.g., one containing multiple ‘if-else‘ branches, ‘case‘ statements,

and sensitive to both clock and asynchronous reset), we decompose it into multiple "minimal basic blocks" based on the internal dataflow dependencies. Each basic block is functionally cohesive, such as the assignment logic for a single register, a state transition condition judgment, or the combinational logic for an output signal.

3.2.2 Verilog Optimization Module

This module parses the abstract syntax tree (AST) of Verilog to identify all ‘always‘ blocks. For each complex ‘always‘ block (e.g., one containing multiple ‘if-else‘ branches, ‘case‘ statements, and sensitive to both clock and asynchronous reset), we decompose it into multiple "minimal basic blocks" based on the internal dataflow dependencies. Each basic block is functionally cohesive, such as the assignment logic for a single register, a state transition condition judgment, or the combinational logic for an output signal.

3.2.3 Verilog Merge Module

The optimized basic blocks need to be reassembled into ‘always‘ blocks that conform to the original design structure to ensure compatibility with the original interface. This module analyzes the control flow of the original block, introduces necessary auxiliary signals or re-adjusts logic priorities, and "stitches" the optimized basic blocks together to form a complex block that is logically equivalent but potentially structurally superior. After merging, the simulation tool is invoked again for equivalence checking to ensure no functional errors were introduced by the optimization.

3.3 Automated Verification Environment Generation (RTL Testbench Generation)

3.3.1 RTL-Spec-Analyzer

This module first parses the RTL code to extract the module's interface list (port names, direction, bit-width). Secondly, it processes user-provided design specifications (e.g., Markdown files), employing natural language processing techniques (combined with an LLM) to extract key timing constraints (such as handshake protocols, reset methods) and critical internal path information. Finally, it structures this information and outputs a "design metadata" document.

3.3.2 Testbench-Generator

Based on the "design metadata" generated in the previous step, this module utilizes the Claw Agent to generate a foundational testbench. This testbench automatically includes: clock and reset signal generators, random stimulus drivers based on the ports, and a basic self-checking logic (scoreboard) framework for capturing outputs and comparing them against expected values. Crucially, it can automatically generate "comparative test cases" for the pre- and post-optimization RTL versions, enabling automated logic equivalence verification by simulating both versions simultaneously and comparing outputs cycle by cycle.

3.4 Agent-Based Automated Flow Integration (Agentic Workflow)

The core of RTL-CLAW lies in the agent's dynamic management of the flow. A typical automated workflow is as follows:

1. nitialization: The user submits the RTL file and design constraints via the OpenClaw interface.
2. Pre-Optimization Verification: The agent invokes the ‘testbench-generator‘ and ‘iverilog‘ to perform functional verification on the original RTL. If simulation fails, the agent attempts to analyze the error log and either rolls back or adjusts its strategy.
3. Pre-Optimization Verification: The agent invokes the ‘testbench-generator‘ and ‘iverilog‘ to perform functional verification on the original RTL. If simulation fails, the agent attempts to analyze the error log and either rolls back or adjusts its strategy.
4. Equivalence Verification: After optimization, the agent invokes ‘iverilog‘ again, using the generated comparative test cases to verify the logical equivalence between the optimized RTL and the original RTL.
5. Logic Synthesis: Once equivalence verification passes, the agent finally invokes ‘yosys‘ (along with ‘abc‘) to perform logic synthesis, specifying optimization goals (area or speed). After synthesis, the agent parses the synthesis report and provides key metrics (area, frequency) back to the user.

4 Results

4.1 Experimental Evaluation

To evaluate the capability of RTL-CLAW in automating hardware design partitioning, we employed a classic traffic light controller as a representative benchmark. The original monolithic design was specified in a single Verilog file (`traffic.v`). Through the command:

```
Use the verilog-partition module to split /path/to/your/file/traffic.v,
and output the results to /path/to/your/output
```

the tool automatically partitioned the finite-state machine into three functional submodules and a top-level wrapper, resulting in the following structure: `top_traffic_light.v`, `u_block_1.v`, `u_block_2.v`, and `u_block_3.v`. This decomposition separates the state sequencing logic (`u_block_1`), timing control (`u_block_2`), and output decoding (`u_block_3`), thereby improving modularity and maintainability.

The key interfaces of the generated submodules are illustrated below. The state machine block, shown in Listing 4.1, manages the traffic light sequence based on the current count value. The timing control block, shown in Listing 4.1, updates the countdown timer according to the pedestrian request and current light status. The output block, shown in Listing 4.1, simply buffers the internal control signals to the actual outputs.

Listing 1: State machine submodule interface (`u_block_1.v`)

```
module u_block_1 (
    input  wire        clk,
    input  wire [7:0]  cnt,
    input  wire        rst_n,
    output reg         p_green,
    output reg         p_red,
    output reg         p_yellow,
    output reg [1:0]   state
);
```

Listing 2: Timer control submodule interface (`u_block_2.v`)

```
module u_block_2 (
    input  wire        clk,
    input  wire        green,
    input  wire        p_green,
    input  wire        p_red,
    input  wire        p_yellow,
    input  wire        pass_request,
    input  wire        red,
    input  wire        rst_n,
    input  wire        yellow,
    output reg [7:0]   cnt
);
```

Listing 3: Output decoder submodule interface (`u_block_3.v`)

```
module u_block_3 (
    input  wire  clk,
    input  wire  p_green,
    input  wire  p_red,
    input  wire  p_yellow,
    input  wire  rst_n,
    output reg  green,
    output reg  red,
    output reg  yellow
);
```

The top-level module (`top_traffic_light.v`) instantiates these three submodules and maintains the overall external interface, including the pedestrian request and the seven-segment display output. The experimental results confirm that RTL-CLAW can successfully partition realistic finite-state machine designs while preserving functional correctness and maintaining clear hierarchical boundaries.

5 Conclusion

5.1 Summary

This paper proposed RTL-CLAW, an innovative IC design automation framework that deeply integrates an AI agent (OpenClaw) with an open-source EDA toolchain. Through the "divide-and-conquer" Partition-Opt-Merge strategy, we successfully addressed the conservatism of traditional tools when optimizing complex RTL logic blocks, effectively improving design PPA metrics. Simultaneously, we implemented an automated verification environment setup. Leveraging intelligent agent scheduling, we connected RTL verification, optimization, and synthesis into a closed-loop process capable of autonomous decision-making. Experimental results demonstrate that RTL-CLAW not only achieves superior design quality but also significantly shortens design iteration cycles, showcasing the immense potential of AI agents in transforming chip design flows.

5.2 Future Work

In the future, we plan to explore the following directions:

1. ntegration into 3DIC Design: Investigating how to extend the RTL-CLAW framework to three-dimensional integrated circuit (3DIC) design, addressing challenges such as logic partitioning across chiplets, timing convergence, and interface standardization.
2. Extension to Physical Design: Deeply integrating OpenClaw's capabilities with open-source physical design tools (e.g., OpenROAD) to achieve an AI-driven full flow from RTL to GDSII, enabling the agent to participate in lower-level physical implementation decisions such as placement, routing, and clock tree synthesis.
3. Enhanced Natural Language Understanding: Enhancing the agent's direct comprehension of natural language design specifications through fine-tuning or prompt engineering, working towards the ultimate automation vision of moving from a "single-sentence design specification" to a "GDSII file," truly lowering the barrier to chip design.

A Appendix Example

This appendix section can hold supplementary figures, extra derivations, interview questions, survey forms, or implementation details.